# EduPredict

Detailed project learning guide

A practical tour of a full-stack student performance prediction system built with a backpropagation neural network, FastAPI, React, and Vite.

| Project area | What you will learn |
| --- | --- |
| Machine learning | Feature preparation, scaling, neural-network inference, thresholds, and calibration |
| Backend | FastAPI routes, validation, artifact loading, batch prediction, and analysis services |
| Frontend | React navigation, API calls, CSV workflows, analytics, and report downloads |
| Operations | PowerShell startup, Docker Compose, health checks, and troubleshooting |

**How to use this guide:** Read Sections 1-4 first for the system mental model. Then run the application and follow the exercises in Section 10 while keeping the API documentation open.

# 1. Project at a glance

EduPredict predicts whether a student is likely to pass. The user enters academic, behavioural, and subject-score information in a React dashboard. The browser sends a JSON payload to a FastAPI service. The service maps the payload into the exact feature layout expected by the trained model, scales the values, runs the Keras model, and returns a probability plus a Pass/Fail label.

The label is a decision aid, not a guaranteed outcome. The probability is learned from historical data and is affected by data quality, feature coverage, class imbalance, and distribution shift.

## Main capabilities

- Single-student prediction and confidence display.
- Batch prediction and CSV import for multiple records.
- Analytics, cohort summaries, calibration information, and model metadata.
- Sensitivity analysis and single-field counterfactual experiments.
- Network visualization and forward-pass traces for interpretability.
- PDF session reports generated from dashboard results.

# 2. Architecture and request flow

| Layer | Location | Responsibility |
|---|---|---|
| Browser UI | frontend/src | Forms, navigation, charts, CSV selection, local session state, and downloads |
| HTTP API | backend/main.py | Routes, request models, error translation, CORS, and orchestration |
| Inference | backend/inference.py | Load artifacts, map fields, impute missing data, scale inputs, predict |
| Analysis | backend/labs.py and backend/meta.py | Sensitivity, counterfactuals, cohort statistics, calibration, metadata |
| Artifacts | model.h5 and scaler.pkl | Trained Keras model and preprocessing/training metadata |
| Reference data | student_data.csv | Small dataset used for cohort statistics and local analysis |

## Typical prediction flow

```
User enters values
        |
        v
React App.jsx -> POST /predict
        |
        v
FastAPI validates PredictRequest
        |
        v
map_frontend_payload() fills the trained feature vector
        |
        v
_features_to_matrix() creates engineered features and scales values
        |
        v
Keras model.predict() -> probability
        |
        v
threshold comparison -> Pass or Fail
        |
        v
JSON response -> dashboard cards and charts
```

# 3. Repository tour

| Path | Why it matters |
|---|---|
| backend/main.py | FastAPI app, request schemas, routes, CORS, and error handling |
| backend/inference.py | Central source of truth for loading and using model artifacts |
| backend/labs.py | Local sensitivity, counterfactuals, cohort comparison, and study tools |
| backend/meta.py | Artifact versioning, calibration bins, completeness, and goal seeking |
| backend/csv_import.py | CSV parsing, column matching, validation, and batch import |
| backend/report_pdf.py | Runtime PDF session-report generation |
| frontend/src/App.jsx | Main dashboard shell and application-level state |
| frontend/src/LabsPage.jsx | Innovation and what-if analysis screens |
| frontend/src/AnalyticsCharts.jsx | Analytics visualizations |
| frontend/src/apiRoutes.js | Development proxy and production API-origin handling |
| scripts/train_bpnn_1m.py | Training, evaluation, threshold calibration, and artifact export |
| start.ps1 | Starts backend and frontend in separate PowerShell windows |
| Dockerfile and docker-compose.yml | Containerized API and production web deployment |

### Important design principle

The frontend should not reproduce model preprocessing. It sends user-facing fields to the API, while backend/inference.py owns mapping, imputation, feature engineering, scaling, and threshold logic. Keeping preprocessing in one place prevents training and serving from silently using different transformations.

# 4. The machine-learning pipeline

The training script uses a binary target named **pass_fail**. It selects numeric input columns while excluding the target and several outcome-adjacent columns that could leak the answer. Leakage would produce impressive validation scores without representing a fair prediction made before the outcome is known.

## Training stages

- Load the CSV and identify feature columns.
- Impute missing numeric values using training-derived values.
- Split data into training, validation, and test sets.
- Fit a MinMaxScaler on training data and reuse it at inference time.
- Train a dense Keras network with ReLU hidden layers and sigmoid output.
- Use dropout, batch normalization, L2 regularization, AdamW, and early stopping.
- Evaluate with accuracy, balanced accuracy, Matthews correlation, AUC, and a confusion matrix.
- Choose a decision threshold on validation data and save metadata with the scaler artifact.

## Model shape

```
Input features
      |
Gaussian noise (optional)
```

```
      |
Dense(h1=128, ReLU) -> BatchNorm -> Dropout
      |
Dense(h2, ReLU)     -> BatchNorm -> Dropout
      |
Dense(h3, ReLU)     -> BatchNorm -> Dropout
      |
Dense(1, sigmoid) = pass probability in [0, 1]
```

A sigmoid output is appropriate for a binary probability-like score. The application then compares that score with a stored decision threshold. The threshold need not be 0.50: it can be calibrated for the selected metric and the cost of false positives versus false negatives.

# 5. Backend deep dive

## FastAPI application

backend/main.py defines the FastAPI instance and Pydantic request models. The PredictRequest model accepts the user-facing fields and ignores unknown extras. Routes call focused service functions rather than embedding model mathematics in the HTTP layer.

```
@app.post("/predict", response_model=PredictResponse)
def predict(body: PredictRequest):
    if not artifacts_exist():
        raise HTTPException(status_code=503, detail="Model artifacts missing.")
    return predict_from_student_row(
        map_frontend_payload(_sparse_payload(body))
    )
```

## Inference and artifacts

- **artifacts_exist()** checks that model.h5 and scaler.pkl are present.
- **load_artifacts()** loads them lazily and caches them in module-level variables.
- **map_frontend_payload()** aligns form keys to training feature columns and applies stored imputation values.
- **_features_to_matrix()** creates any legacy engineered columns, orders features, and calls scaler.transform().
- **predict_from_student_row()** returns probability, complement probability, label, and threshold.

## Why lazy loading matters

The server can start and expose lightweight metadata routes without immediately loading TensorFlow. The model is loaded on the first inference request and reused afterward, reducing repeated disk and initialization cost.

## Analysis endpoints

| Endpoint | Learning concept |
|---|---|
| POST /labs/sensitivity | Finite differences: perturb one input slightly and observe delta in probability |
| POST /labs/counterfactual | What-if reasoning: compare baseline with a chosen field value |
| GET /labs/cohort/summary | Descriptive statistics from reference data |
| GET /meta/calibration | Compare predicted probability bins with empirical pass rates |
| POST /meta/completeness | Measure how much of the expected input was supplied |
| POST /predict/trace | Inspect activations across dense layers |

# 6. Frontend deep dive

The React app is organized as a dashboard shell with navigation for prediction, student records, CSV imports, reports, analytics, improvement coaching, timetable planning, settings, help, and innovation labs. API helper modules keep endpoint construction separate from UI components.

## Frontend responsibilities

- Collect user input and show validation or loading states.
- Call the backend instead of calculating model outputs in the browser.
- Render prediction cards, charts, tables, traces, and experiment results.
- Keep a local audit/session history for the dashboard experience.

- Send CSV files as multipart form data to the server-side importer.
- Download generated PDF reports from the backend response.

# 7. Running and operating the project

## Local setup

```
python -m pip install -r backend\requirements.txt
Set-Location frontend
npm install
Set-Location ..
.\start.ps1
```

The launcher starts FastAPI at port 8000 and Vite at port 5173. During development, Vite proxies /api requests to http://127.0.0.1:8000. Open /docs to explore and execute backend routes interactively.

## Health verification

```
Invoke-RestMethod http://127.0.0.1:8000/health

# Expected shape:
# {"status": "ok", "artifacts_present": true}
```

## Docker deployment

Docker Compose builds the API from the root Dockerfile and the web container from frontend/Dockerfile. The API is exposed on 8000 and the production web server on 8080. Model artifacts and reference data are mounted read-only into the API container.

```
docker compose up --build
```

## Common troubleshooting

| Symptom | What to check |
| --- | --- |
| 503 model artifacts missing | Confirm model.h5 and scaler.pkl are in the project root |
| Frontend cannot reach API | Confirm port 8000 is listening and inspect Vite proxy settings |
| TensorFlow import failure | Use a compatible Python version and reinstall backend requirements |
| CSV columns rejected | Compare headers with the feature names accepted by PredictRequest |
| Slow first prediction | Expected: TensorFlow loads lazily on the first inference request |
| Probability looks surprising | Inspect imputation, feature scale, calibration, and distribution shift |

# 8. API learning examples

## Single prediction

```
$body = @{
  attendance_rate = 0.90
  assignment_avg = 82
  previous_gpa = 3.2
  study_hours_daily = 2.5
  sleep_hours = 7
  mental_stress = 3
  parent_involvement = 7
} | ConvertTo-Json

Invoke-RestMethod `
  -Method Post `
  -Uri http://127.0.0.1:8000/predict `
  -ContentType "application/json" `
  -Body $body
```

Study the response fields: prediction is the thresholded label, pass_probability is the model score, fail_probability is its complement, and decision_threshold explains how the label was chosen.

## Batch prediction

```
$body = @{
  rows = @(
    @{ attendance_rate = 0.95; assignment_avg = 90; study_hours_daily = 3 },
    @{ attendance_rate = 0.60; assignment_avg = 55; study_hours_daily = 0.5 }
  )
} | ConvertTo-Json -Depth 5

Invoke-RestMethod -Method Post `
  -Uri http://127.0.0.1:8000/predict/batch `
  -ContentType "application/json" -Body $body
```

# 9. Responsible interpretation

A predictive score should support conversations and targeted support, not replace teacher judgment or become an automatic high-stakes decision. The project includes interpretability tools, but sensitivity is local and model-dependent: it describes how the score changes near one particular input, not a causal effect.

## Questions to ask when interpreting a result

- Were important fields missing and therefore imputed?
- Are the input units and ranges correct?
- Does the student resemble the training population?
- Is the probability calibrated for this subgroup and time period?
- Could a proxy feature encode unfair or irrelevant differences?
- What supportive action would be useful regardless of the exact label?

# 10. Suggested learning path and exercises

| Stage | Exercise | Outcome |
|---|---|---|
| 1. Orientation | Run the app, open /docs, and make one /health request | Understand the two-service architecture |
| 2. API basics | Send two /predict requests with different inputs | See probability and threshold behaviour |
| 3. Mapping | Remove optional fields and inspect the response | Understand imputation and sparse payloads |
| 4. Analysis | Call sensitivity and counterfactual endpoints | Distinguish local association from causality |
| 5. Frontend | Trace the predictor form to its Axios call | Connect UI state to an API response |
| 6. Data | Import a small CSV and compare cohort summary | Learn batch workflows and validation |
| 7. ML | Read train_bpnn_1m.py and identify leakage exclusions | Understand a reproducible training pipeline |
| 8. Experiment | Change a UI label or add a harmless metadata field | Practice full-stack change and verification |

## Mini-project ideas

- Add a clearly labelled model version card using /meta/version.
- Plot calibration bins and compare empirical pass rate with predicted probability.
- Add input-range validation with accessible error messages.
- Create a test for payload mapping and the decision-threshold boundary.
- Add a data-quality report showing missing fields before prediction.
- Compare a simple logistic-regression baseline with the BPNN.

# 11. Quick reference

| Task | Command or URL |
|---|---|
| Start locally | .\start.ps1 |
| Frontend | http://localhost:5173 |

| Task | Command or URL |
| --- | --- |
| API docs | http://localhost:8000/docs |
| Health | http://localhost:8000/health |
| Build frontend | Set-Location frontend; npm run build |
| Lint frontend | Set-Location frontend; npm run lint |
| Run containers | docker compose up --build |
| Train model | python scripts\train_bpnn_1m.py |

Keep the root README.md nearby for the short operational instructions. This PDF is the expanded learning companion: use it to understand why the files exist and how the pieces work together.